



Objective

In this assignment, you will implement two sequence-to-sequence (seq2seq) models based on recurrent networks and the Transformer architecture to build a neural machine translation (NMT) systems translating from French to English.

Dataset

You will use a dataset consisting of parallel French and English sentences and partitioned into training, validation, and test splits. You will need to tokenize the data using French and English tokenizers, which are provided with the assignment resources. You will use BPE (byte pair encoding) tokenization, which can split a less common word into multiple subword tokens. The dataset and tokenizers are available at this link.

Part 1. Transformer-Based Architecture

The model you will be implementing is an encoder-decoder Transformer model. For simplicity, we made several modifications to the architecture described in this paper:

- Use learned positional embeddings instead of sinusoidal positional embeddings.
- No dropout in the embedding layer and the attention weights.
- Use weight tying between the decoder's input embeddings and the output projection matrix.

We will describe each block of the model. Let d be the embedding dimension of input embeddings and hidden states, N the maximum sequence length, $V^{(e)}$ and $V^{(d)}$ the vocab sizes for encoder and decoder vocabulary, respectively. Let n be the length of a particular input sequence. In practice, the model will process B sequences in a batch in parallel, and the sequence might contain pad tokens, which should be excluded from the attention mechanism and the loss function.

- **Embedding:** Let $E^{(e)} \in R^{V^{(e)} \times d}$ be the token embedding tables and $P^{(e)} \in R^{N \times d}$ be the positional embedding tables for the encoder. To embed a token with token id t at position i , we look up the token embedding at index t and position embedding at index i and add the two embeddings. The same procedure is done in the decoder embedding layer with separate matrices $E^{(d)} \in R^{V^{(d)} \times d}$ and $P^{(d)} \in R^{N \times d}$. The output of each embedding layer is a sequence of vectors $h_1^{(0)}, h_2^{(0)}, \dots, h_n^{(0)} \in R^d$.
- **Multi-head attention:** The input to this layer is a sequence of hidden state vectors from the previous layer $h_1^{(l-1)}, h_2^{(l-1)}, \dots, h_n^{(l-1)} \in R^d$. We project each of these vectors to query, key, and value vectors:

$$q_i^{(l)} = W^{(q)} h_i^{(l-1)} \quad k_i^{(l)} = W^{(k)} h_i^{(l-1)} \quad v_i^{(l)} = W^{(v)} h_i^{(l-1)}$$

where $q_i^{(l)}, k_i^{(l)}, v_i^{(l)} \in R^d$. For the sake of clarity, we are omitting the layer index (l) from the weight matrices and intermediate variables. In cross-attention, the input key and value matrices would be the output states of the encoder, whereas the queries would be computed from the hidden states of the decoder. We then split these vectors into H separate vectors, where H is the number of attention heads:

$$q_i = \begin{bmatrix} q_{i,1} \\ q_{i,2} \\ \vdots \\ q_{i,H} \end{bmatrix}, \quad k_i = \begin{bmatrix} k_{i,1} \\ k_{i,2} \\ \vdots \\ k_{i,H} \end{bmatrix}, \quad v_i = \begin{bmatrix} v_{i,1} \\ v_{i,2} \\ \vdots \\ v_{i,H} \end{bmatrix}$$

where $q_{i,h}, k_{i,h}, v_{i,h} \in R^{d_H}$ and $d_H = \frac{d}{H}$ is the head dimension. For each head h , we compute a scaled dot product attention:

$$y_{i,h} = \sum_{j=1}^n \left(\frac{\exp(q_{i,h} \cdot k_{j,h} / \sqrt{d_H})}{\sum_{j'=1}^n \exp(q_{i,h} \cdot k_{j',h} / \sqrt{d_H})} \right) v_{j,h}$$

where each $y_{i,h} \in R^{d_H}$. We need to make sure to avoid attending to pad tokens; in practice, this is achieved by setting the attention score $(q_i^{(h)} \cdot k_j^{(h)}) / \sqrt{d_H}$ to a very large negative value if there is a pad token at position j . When computing the self-attention in the decoder, we use causal masking:

$$y_{i,h} = \sum_{j=1}^i \left(\frac{\exp(q_{i,h} \cdot k_{j,h} / \sqrt{d_H})}{\sum_{j'=1}^i \exp(q_{i,h} \cdot k_{j',h} / \sqrt{d_H})} \right) v_{j,h}$$

Finally, we stack the attention outputs and project each token to obtain output vectors $h_1^{(l)}, \dots, h_n^{(l)} \in R^d$:

$$h_i^{(l)} = W^{(o)} \begin{bmatrix} y_{i,1} \\ y_{i,2} \\ \vdots \\ y_{i,H} \end{bmatrix}$$

- **Feedforward layers:** The input is $h_1^{(l-1)}, \dots, h_n^{(l-1)} \in R^d$. Each layer learns $W^{(1)} \in R^{d_I \times d}$ and $W^{(2)} \in R^{d \times d_I}$. These project to a larger dimension d_I , apply ReLU, project back, and apply dropout:

$$h_i^{(l)} = \text{Dropout}(W^{(2)} \text{ReLU}(W^{(1)} h_i^{(l-1)}))$$

- **Add & Norm:** After each block, we add a residual connection and perform layer normalization.
- **Output layer:** Next token prediction at position i :

$$\text{logits}_i = E^{(d)} h_i^{(l-1)} \in R^{V^{(d)}}$$

Model Parameters (Transformer)

Parameter	Value
hidden size	32
intermediate size	32×4
num attention heads	4
num encoder layers	3
num decoder layers	3
max sequence length	32
hidden dropout prob	0.1
batch size	32
learning rate	1×10^{-3}
max epochs	10

Part 2. Recurrent Network-Based Architecture

Implementation using a Bidirectional LSTM for the encoder and a Unidirectional LSTM for the decoder with Additive Attention.

- **Encoder (Bi-LSTM):** Concatenates forward ($\vec{h}_i$) and backward ($\overleftarrow{h}_i$) states:

$$h_i = [\vec{h}_i; \overleftarrow{h}_i] \in R^{2h}$$

- **Decoder Initialization:** $s_0 = \tanh(W^{(init)}[\vec{h}_n; \overleftarrow{h}_1]), s_0 \in R^h$.
- **Additive Attention:** $e_{t,i} = v^\top \tanh(W^{(a)}s_{t-1} + U^{(a)}h_i)$.

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^n \exp(e_{t,j})}, \quad c_t = \sum_{i=1}^n \alpha_{t,i} h_i \in R^{2h}$$

- **Decoder Step:** $s_t = \text{LSTM}([y_{t-1}; c_t], s_{t-1})$.
- **Output Layer:** $\text{logits}_t = W^{(out)}[s_t; c_t; y_{t-1}]$.

Model Parameters (LSTM)

Parameter	Value
embed size (d)	256
hidden size (h)	512
encoder bidirectional	True
num layers	1 (or 2)
lstm dropout	0.3
batch size	32
learning rate	1×10^{-3}
max epochs	10

Notes

1. Use `transformers` package only for tokenization.
2. Implement vectorized multi-head attention;
no `nn.MultiheadAttention` or `nn.functional.scaled_dot_product_attention`.
3. Implement the embedding layer (sum of token and positional embeddings).
4. `torch.nn` is allowed for feedforward and normalization.
5. Implement beam search and compute BLEU scores for both models.
6. Visualize attention weights for both models.
7. Implement saving and loading checkpoints.
8. Be prepared to test the model live during the discussion.